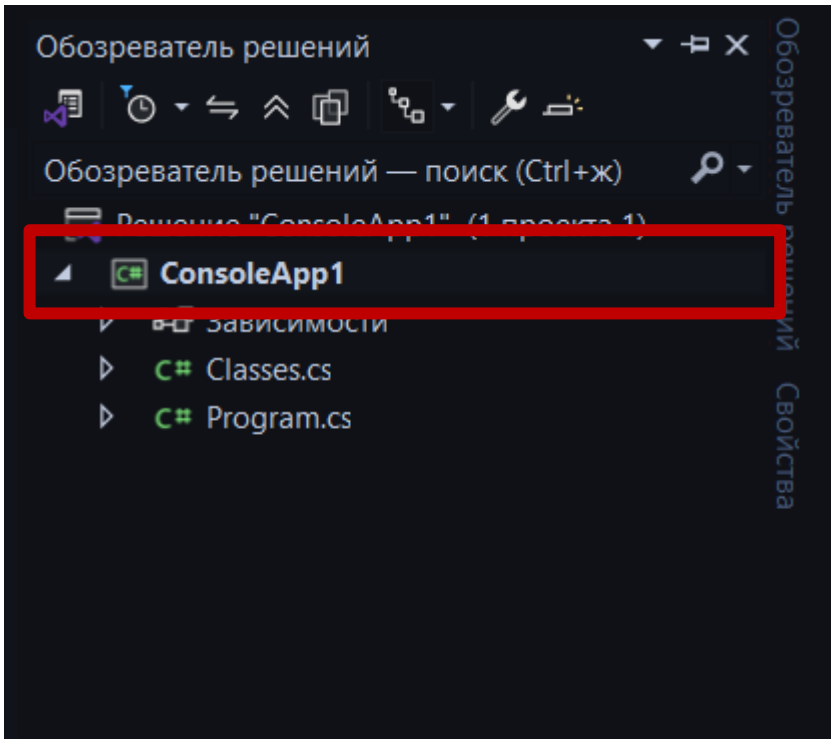
Создание Окна Авторизации

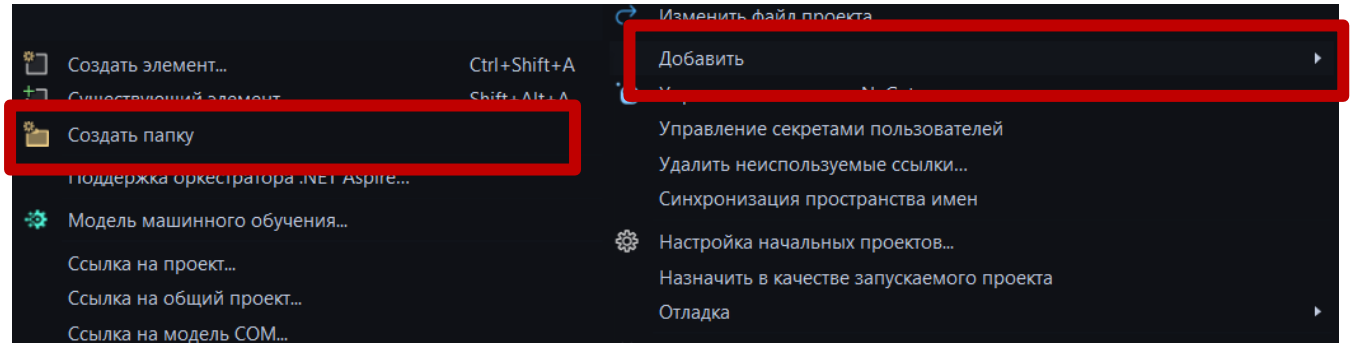
КОГДА СОЗДАЕТЕ ПРОЕКТ ПРОВЕРЬТЕ, ЧТОБЫ НЕ СТОЯЛА ГАЛОЧКА «ПОМЕСТИТЬ РЕШЕНИЕ И ПРОЕКТ В ОДНОМ КАТАЛОГЕ» ИНАЧЕ КОМАНДЫ ENTITY FRAMEWORK НЕ СРАБОТАЮТ

1. В обозревателе решений создайте папку **Windows**

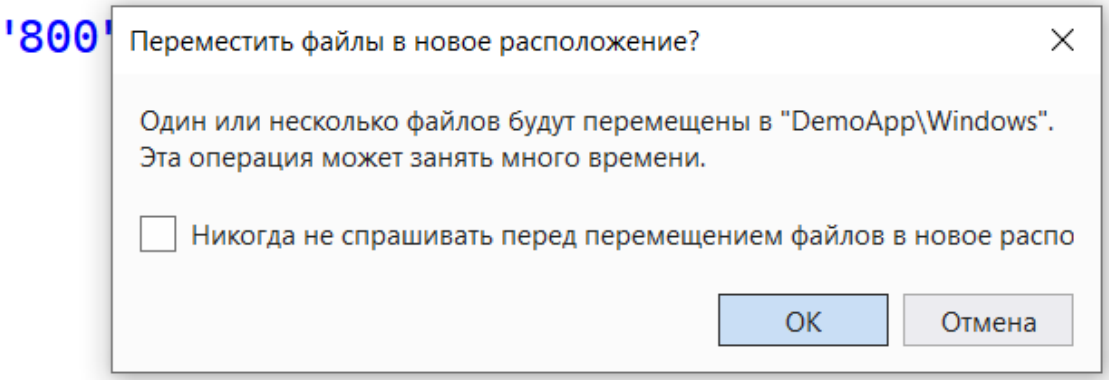
1.1 Откройте обозреватель решений, кликните ПКМ (правой кнопкой мыши по проекту)

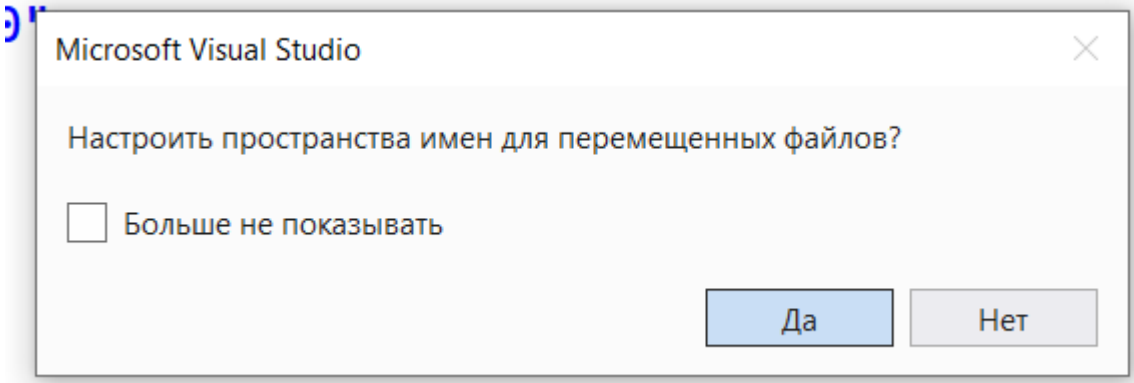


Создайте папку, для этого выберите «Добавить» → «Папку». После чего переименуйте, нажав по папке ПКМ → «Переименовать» или нажав F2, предварительно кликнув на папку.



2. В папку переместите окно MainWindow. Согласитесь со всем предложенным.





2.1 Переименуйте его, например, AuthorizationWindow, нажав по файлу ПКМ → «Переименовать» или нажав F2, предварительно кликнув на файл окна.

2.2 В файле .xaml измените название и пространство окна

```
<Window x:Class="DemoApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:DemoApp"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="800">
```

↓

```
<Window x:Class="DemoApp.Windows.AuthorizationWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:DemoApp"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="800">
```

2.3 Так же сделайте и в файле .xaml.cs

```
namespace DemoApp
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    Ссылок: 1
    public partial class MainWindow : Window
    {
        Ссылок: 0
        public MainWindow()
        {
            InitializeComponent();
```

```
namespace DemoApp.Windows
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    Ссылок: 1
    public partial class AuthorizationWindow   Window
    {
        Ссылок: 0
        public AuthorizationWindow()
        {
            InitializeComponent();
        }
    }
}
```

Также необходимо открыть файл App.xaml и поменять название окна.

```
<Application
    x:Class="DemoApp.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="clr-namespace:DemoApp"
    StartupUri="MainWindow.xaml">
    <Application.Resources>

    </Application.Resources>
</Application>
```

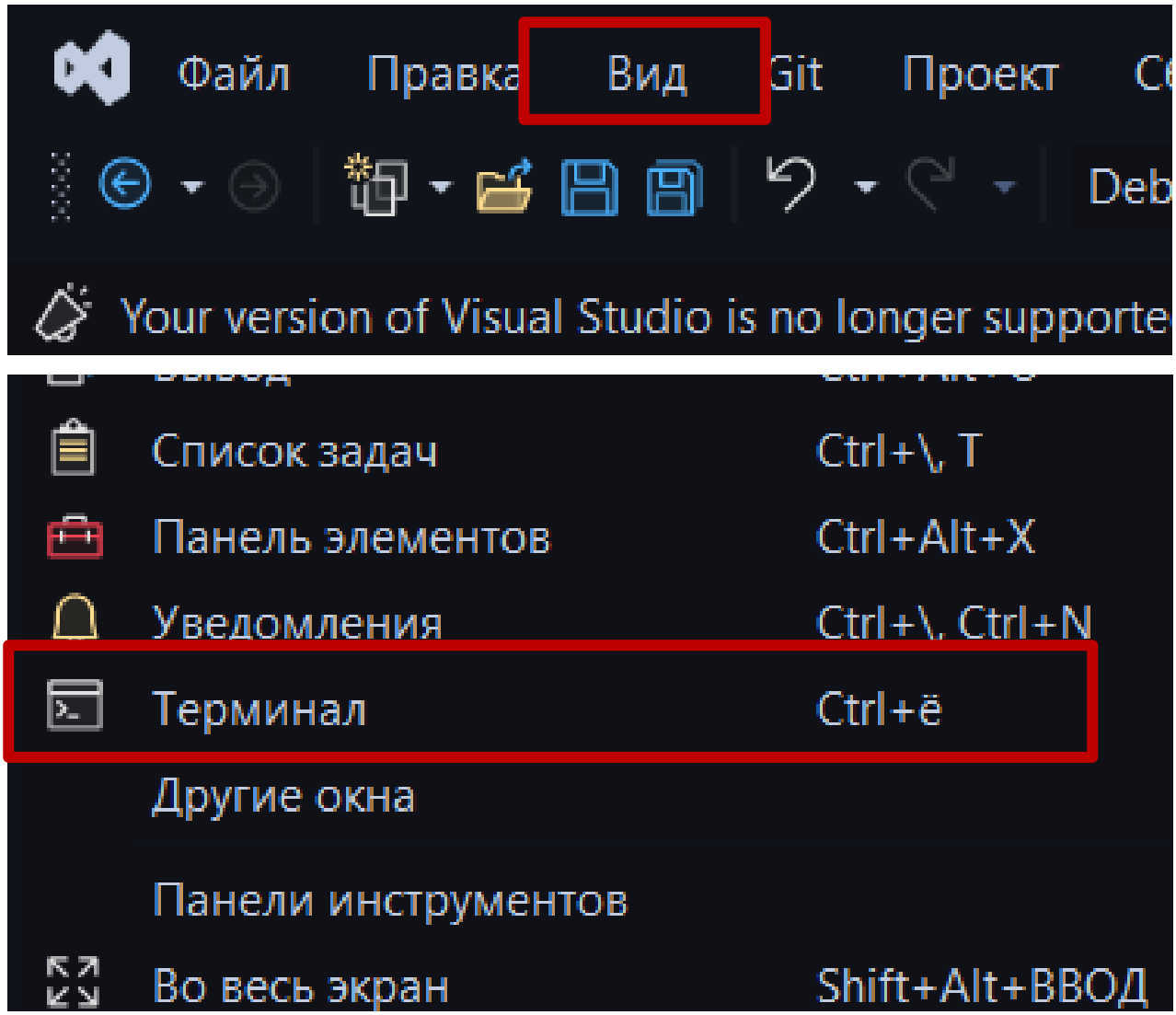
```
<Application
    x:Class="DemoApp.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="clr-namespace:DemoApp"
    StartupUri="/Windows/AuthorizationWindow.xaml">
    <Application.Resources>

    </Application.Resources>
</Application>
```

Сохраните все файлы (ctrl + shift + S) и перезайдите в VS.

3. Добавьте необходимые пакеты EntityFramework и подключите базу данных.

3.1 Откройте PowerShell (комбинация клавиш ctrl + ` (ctrl + Ё) или «Вид» → «Терминал»)



3.2 Введите необходимые команды для установки пакетов

```
dotnet add package Microsoft.EntityFrameworkCore --version 8.0.11
dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.11
dotnet add package Pomelo.EntityFrameworkCore.MySql --version 8.0.2
```

**** ПРИМЕЧАНИЕ: НЕОБЯЗАТЕЛЬНО КАЖДЫЙ РАЗ ПРОПИСЫВАТЬ КОМАНДЫ. ПРОПИШИТЕ ОДНУ КОМАНДУ, А ПОСЛЕ ВЫПОЛНЕНИЯ НАЖМИТЕ СТРЕЛОЧКУ ВВЕРХ (↑) И ДОБАВЛЯЙТЕ ИЛИ ИЗМЕНЯЙТЕ КОМАНДЫ**

**** УПРАВЛЕНИЕ В КОНСОЛИ ПРОИЗВОДИТСЯ ТОЛЬКО С ПОМОЩЬЮ КЛАВИАТУРЫ**

3.3 После пропишите команду для подключения БД

```
dotnet ef dbcontext scaffold "строка соединения с базой данных" Pomelo.EntityFrameworkCore.MySql --context-dir DbContexts --output-dir Entities
```

**** КАВЫЧКИ ОСТАЮТСЯ**

**** ВМЕСТО *СТРОКА СОЕДИНЕНИЯ С БАЗОЙ ДАННЫХ* ВЫ ПРОПИСЫВАЕТЕ**

```
server=localhost; user= root; password=123456789; database=название_бд
```

**** ВМЕСТО *НАЗВАНИЕ_БД* ВЫ ПРОПИСЫВАЕТЕ СВОЮ СОЗДАННУЮ БД**

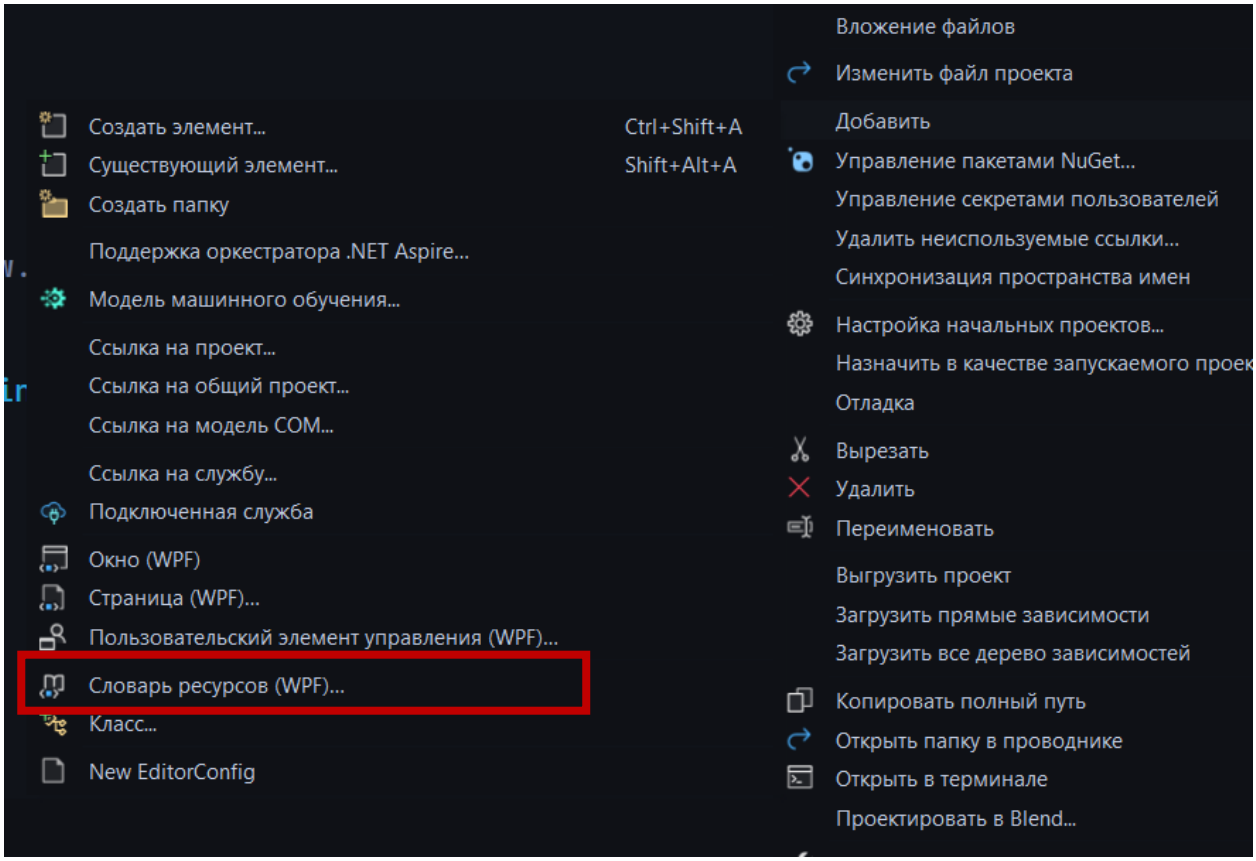
РАЗМЕТКА ОКНА

Для удобства реализации рекомендуется прописать стили для кнопок и шрифта.

ДОБАВЛЕНИЕ СТИЛЕЙ

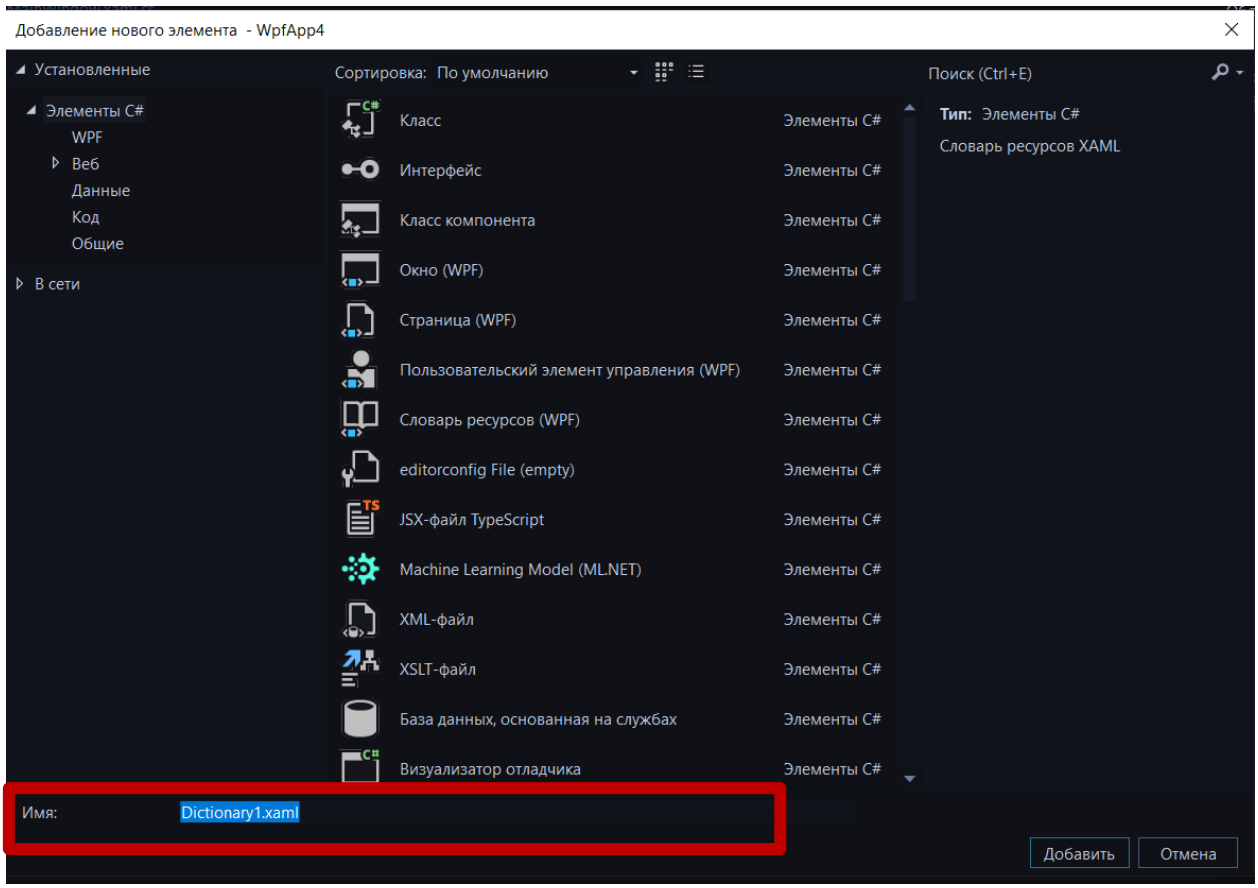
Нужно создать словарь ресурсов и ссылаться на него в **App.xaml**

Для этого ПКМ по проекту → «Добавить» → «Словарь ресурсов»



В появившемся окне пропишите название словаря

Можно переписать полностью название без расширения, VS сам допишет его. **(НЕ ОСТАВЛЯЙТЕ Dictionary1 И ТД, ДАВАЙТЕ ПРАВИЛЬНЫЕ НАЗВАНИЯ ДЛЯ ФАЙЛОВ И КЛАССОВ)**

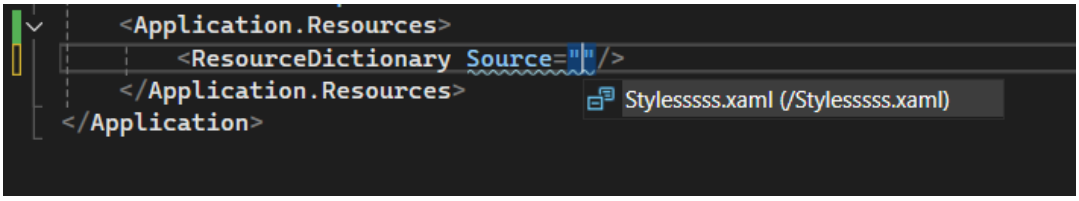


В App.xaml пропишем

```
<Application.Resources>
   <ResourceDictionary Source="/Stylesssss.xaml"/>
</Application.Resources>
```

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СЕБЯ НАЗВАНИЕ СТИЛЯ ВЫ УКАЗЫВАЕТЕ ТАКОЕ ЖЕ, КАКОЕ УКАЗАЛИ ПРИ СОЗДАНИИ

Visual Studio предложит выбрать словарь ресурсов



Для применения стиля ко всем элементам **НЕ** прописывайте ключ (x:Key). Ключ можно прописать для кнопки. Необходимо создать стили для:

– TextBlock, указав название шрифта (FontFamily), размер шрифта (FontSize), внешние отступы (Margin)

```
<Style TargetType="TextBlock">
    <Setter Property="FontFamily" Value="Times New Roman" />
    <Setter Property="FontSize" Value="14" />
    <Setter Property="Margin" Value="5" />
</Style>
```

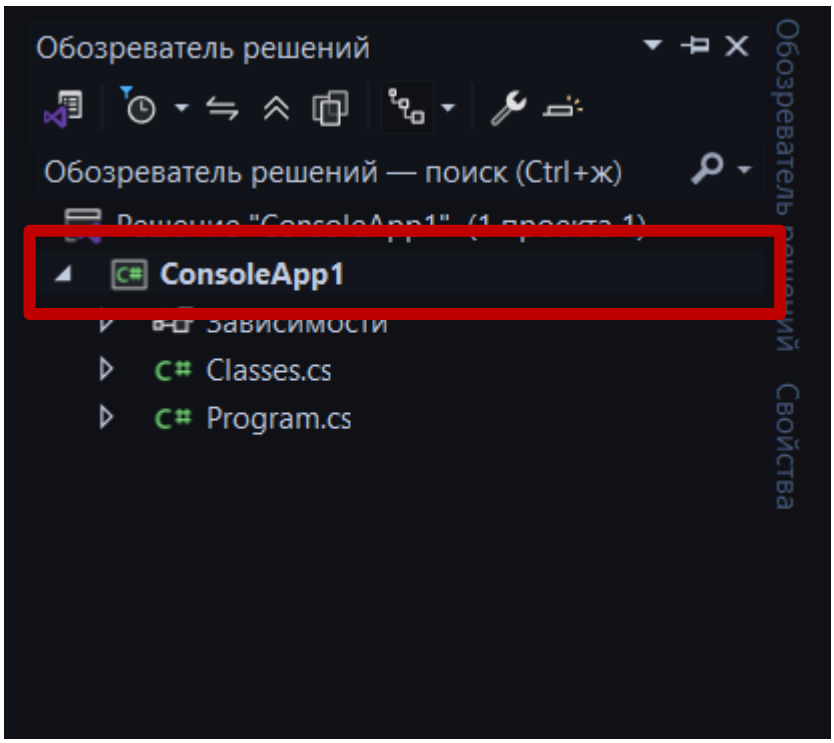
!!!ПРИМЕР!!! ПОМНИТЕ, ЧТО ВСЕ ШРИФТЫ И ЦВЕТА УКАЗАНЫ В РУКОВОДСТВЕ ПО СТИЛЮ И МОГУТ ОТЛИЧАТЬСЯ

– TextBox, установите те же параметры, их можно скопировать из стиля TextBlock’a

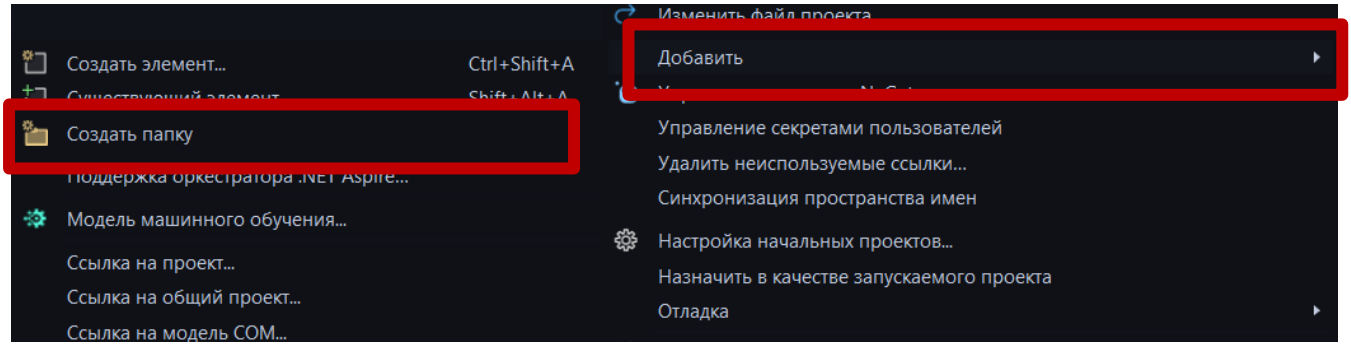
– Button, также скопируйте свойства для шрифта, а также добавьте внутренние отступы (Padding), уберите границы (можно использовать свойство BorderThickness со значением 0), установите ширину шрифта (FontWeight), а также установите фон (Background), цвет берется из руководства по стилю. Для кнопки также лучше прописать ключ (x:Key).

1. Добавьте необходимые изображения для иконки и логотипа, можно также сразу импортировать изображения товаров

1.1 Откройте обозреватель решений, кликните ПКМ (правой кнопкой мыши по проекту)

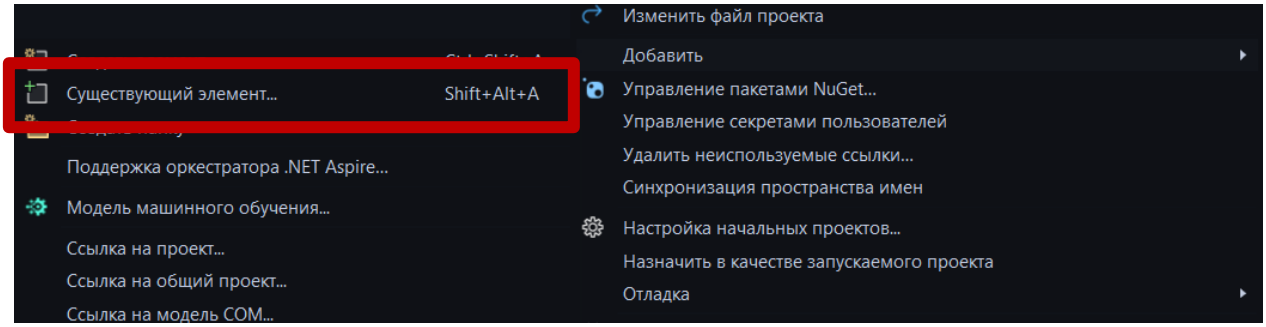


Создайте папку Images, для этого выберите «Добавить» → «Папку»

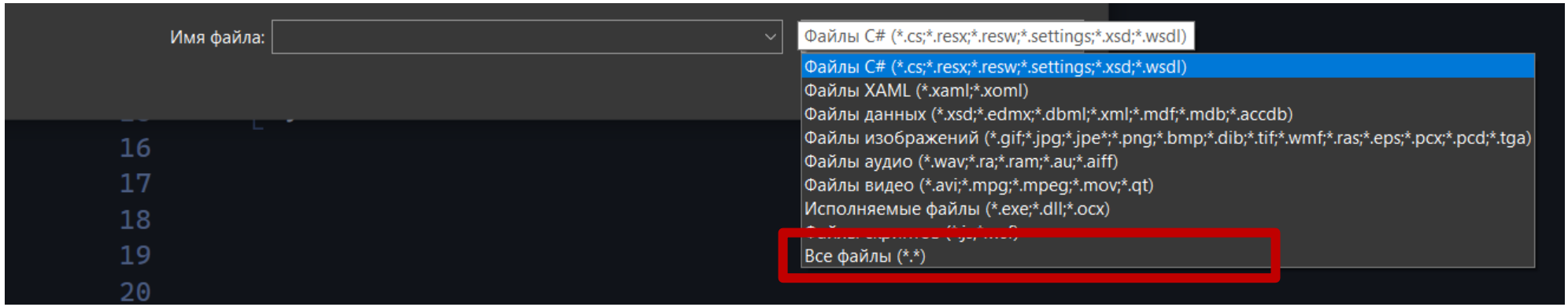


1.2 Поместите туда все необходимые изображения

Так же кликните ПКМ по проекту → Добавить → Существующий элемент

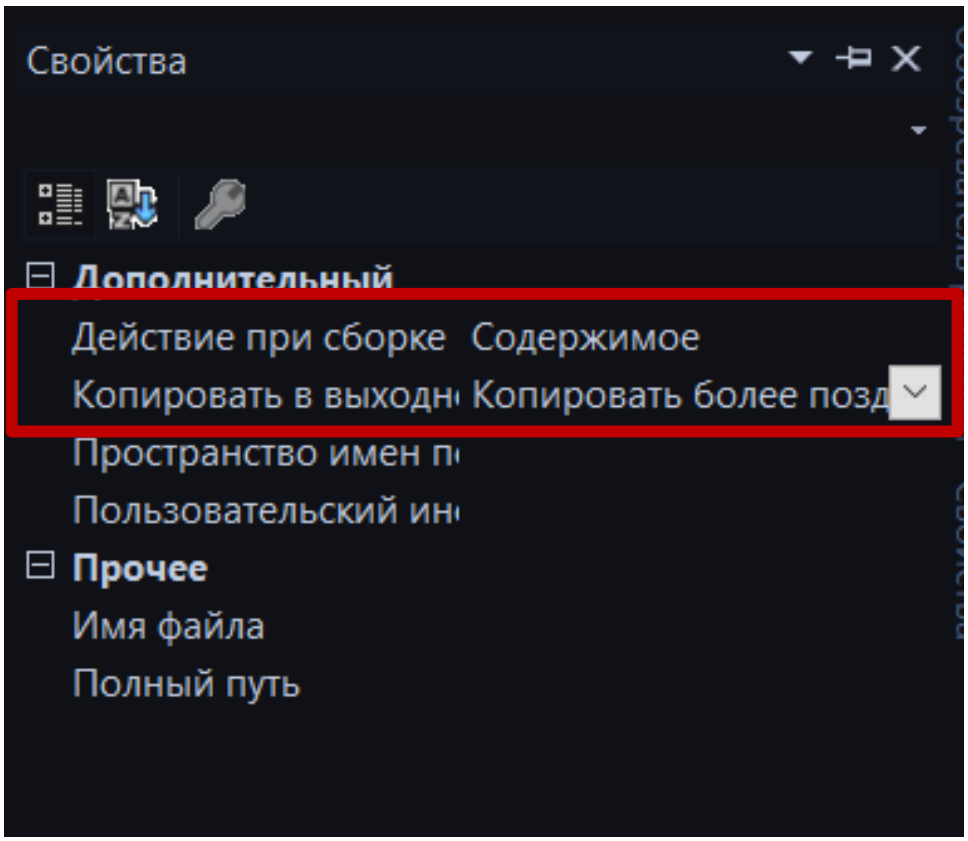


В диалоговом окне установите фильтр «Все файлы» и добавьте необходимые изображения (можно сразу несколько)



1.3 Измените свойства изображений. Для этого выделите несколько изображений (Нажмите на первое изображение, после удерживайте Shift и нажмите на последнее изображение), перейдите в свойства (если этого окна нет, то в верхней панели нажмите Вид → Свойства или F4).

Установите свойство «Действие при сборке» → «Содержимое», «Копировать в выходной каталог» → «Копировать более позднюю версию»



2. Измените название окна (Title) с «MainWindow» на «Авторизация», установите его размеры (Width и Height), иконку (Icon), и разместите начальную позицию (WindowStartUpLocation) в центре, а также ограничьте изменение размеров окна (ResizeMode).

```
Title="Авторизация"
Width="300"
Height="400"
Icon="/Images/Icon.ico"
ResizeMode="NoResize"
WindowStartupLocation="CenterScreen"
```

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СЕБЯ

3. Grid можно заменить на StackPanel, чтобы элементы шли последовательно друг за другом.

```
    <Grid>

    </Grid>
</Window>
```

```
    <StackPanel>

    </StackPanel>
</Window>
```

4. Для StackPanel пропишите выравнивание по центру (HorizontalAlignment и VerticalAlignment)

5. Добавьте логотип компании (Image) Icon.png

6. Разместите TextBlock и в свойстве Text напишите Email или Логин

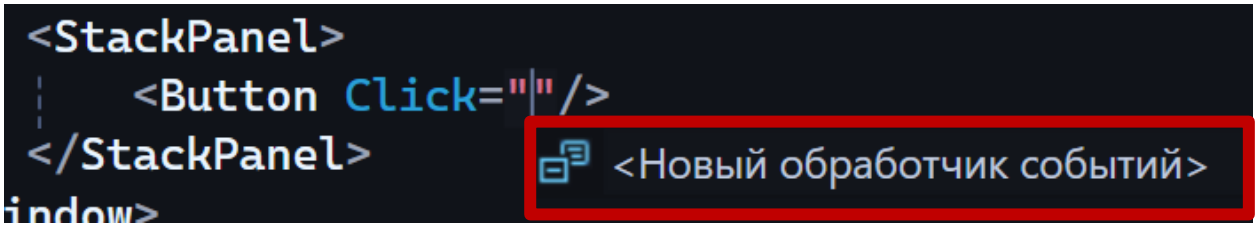
7. Разместите TextBox и присвойте ему имя (x:Name)

8. Разместите TextBlock и в свойстве Text напишите Пароль

9. Разместите PasswordBox и присвойте ему имя (x:Name)

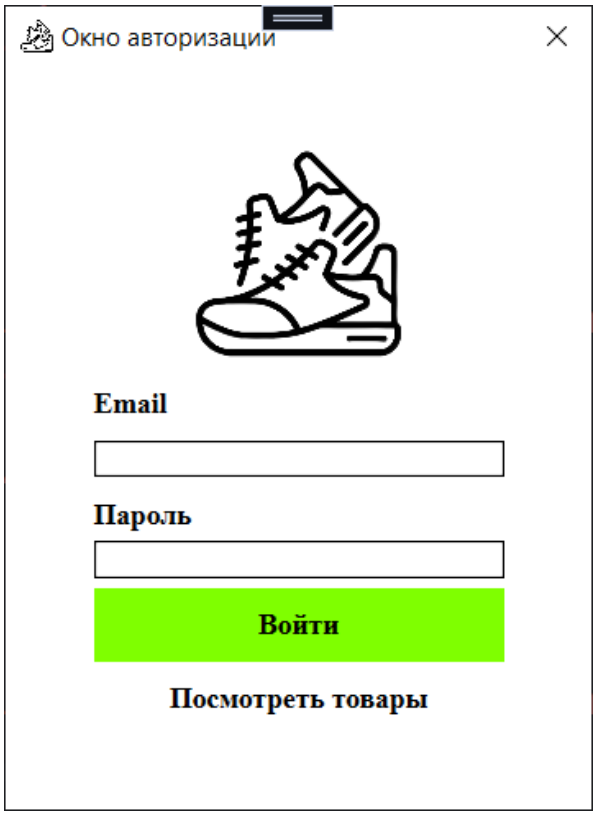
10. Добавьте кнопку для входа, в свойстве Content укажите необходимый текст, присвойте кнопке имя (x:Name), а также создайте обработчик событий (Click), пропишите свойство Cursor и значение Hand

**** НЕ НУЖНО ПРОПИСЫВАТЬ НАЗВАНИЕ МЕТОДА, VS ПРЕДЛОЖИТ СОЗДАТЬ НОВЫЙ ОБРАБОТЧИК СОБЫТИЙ. НАЖМИТЕ TAB. VS АВТОМАТИЧЕСКИ ВПИШЕТ ПРИВЯЗКУ И СОЗДАСТ МЕТОД В C# - ФАЙЛЕ.**



11. Добавьте текстовый блок или кнопку, чтобы можно было просмотреть товары без авторизации в роли гостя. Также присвойте имя (x:Name) и необходимый текст. Для Textblock'а пропишите обработчик событий **MouseLeftButtonUp.** Для кнопки обработчик событий так же остается Click. Пропишите свойство Cursor и значение Hand.

!!!ПРИМЕР!!!

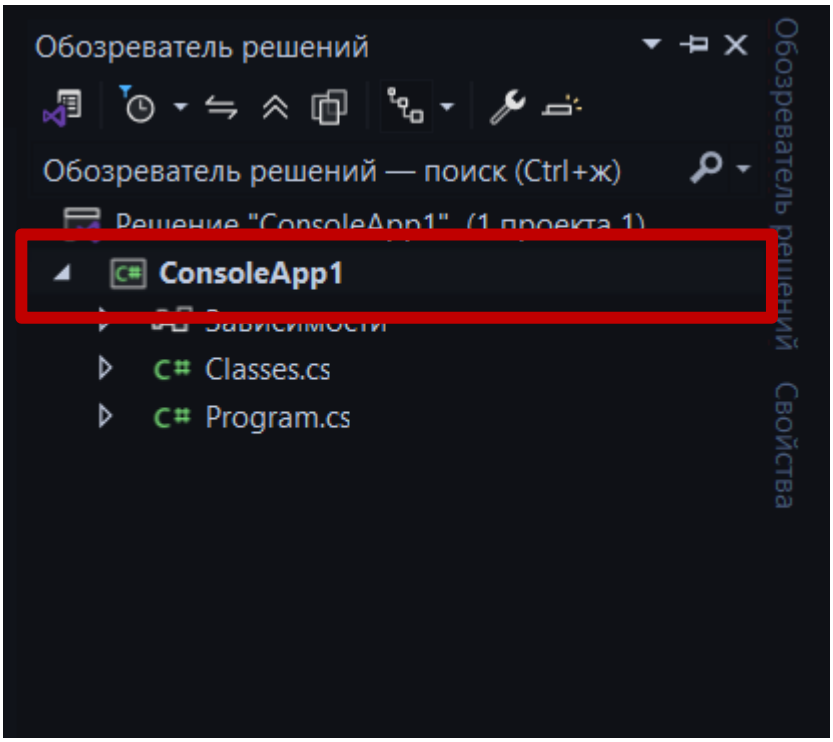


ЛОГИКА ОКНА

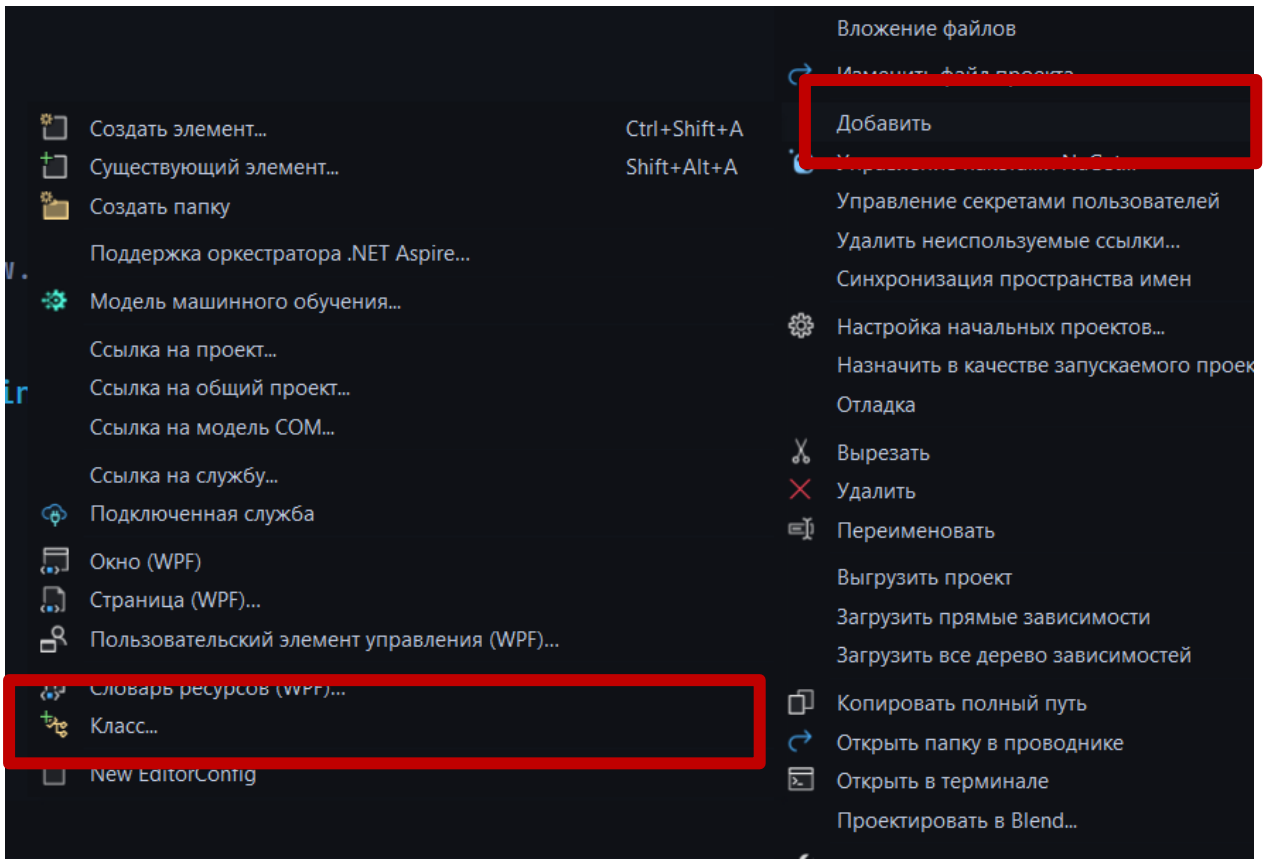
1. Откройте C#-файл (расширение *.xaml.cs) окна авторизации.

2. Для реализации распределения ролей создайте static класс (отдельным файлом) CurrentUser

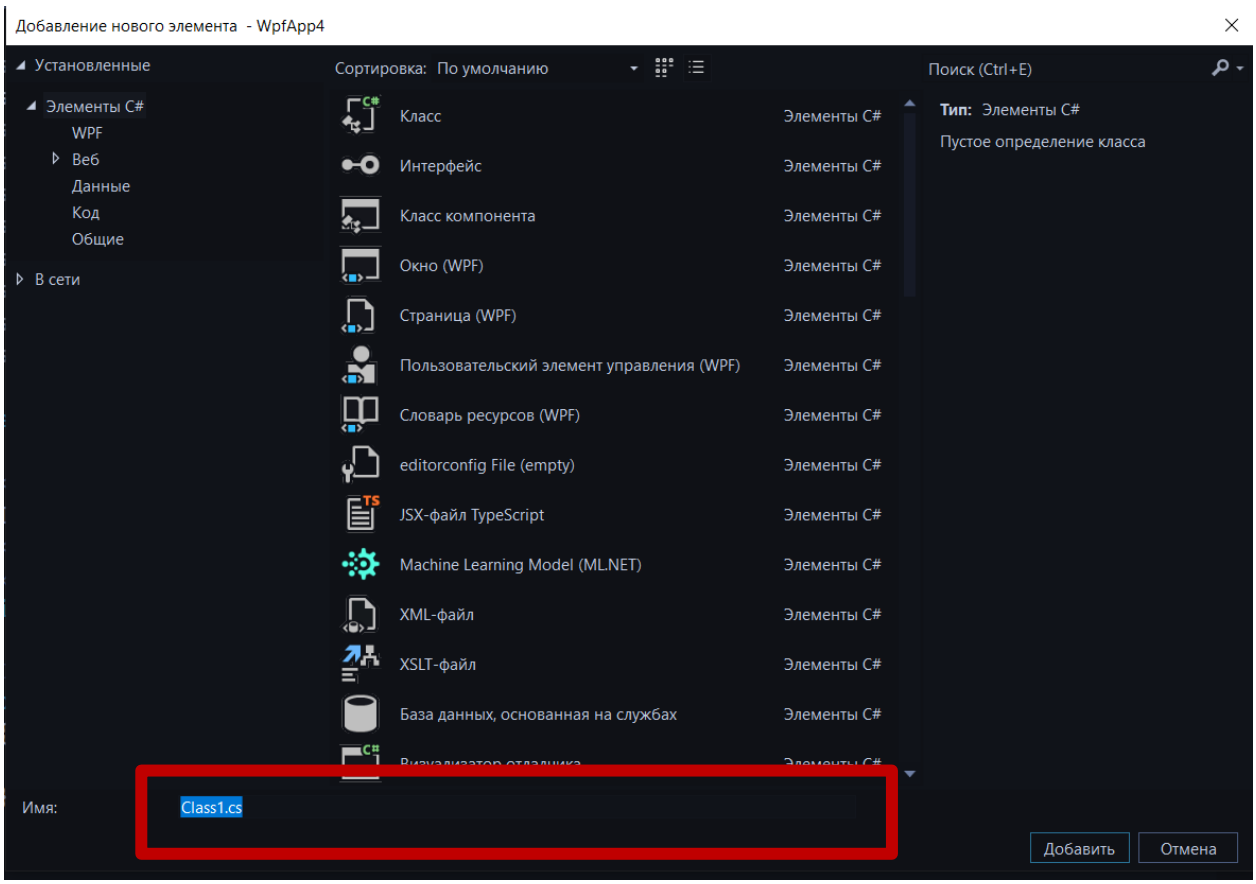
3.1 Откройте обозреватель решений, кликните ПКМ (правой кнопкой мыши по проекту)



3.2 Выберите «Добавить» → «Класс»



3.3 В появившемся окне присвойте название классу **(НЕ ОСТАВЛЯЙТЕ CLASS1 И ТД, ДАВАЙТЕ ПРАВИЛЬНЫЕ НАЗВАНИЯ ДЛЯ ФАЙЛОВ И КЛАССОВ)**



Можно переписать полностью название без расширения, VS сам допишет его. **ПОМНИТЕ,** что для классов используется соглашение об именовании **PascalCase**, это значит, что ***каждое слово начинается с заглавной (большой) буквы.***

3.4 Поменяйте модификатор доступа **internal** на **public**, чтобы класс был доступен и в других классах (файлах, окнах).

```
namespace WpfApp4
{
    Ссылок: 0
    internal class CurrentUser
    {
    }
}
```

```
namespace WpfApp4
{
    Ссылок: 0
    public class CurrentUser
    {
    }
}
```

Не забудьте добавить ключевое слово **static**, чтобы сделать класс статическим (не требуют создания объекта и позволяют обращаться к полям и методам)

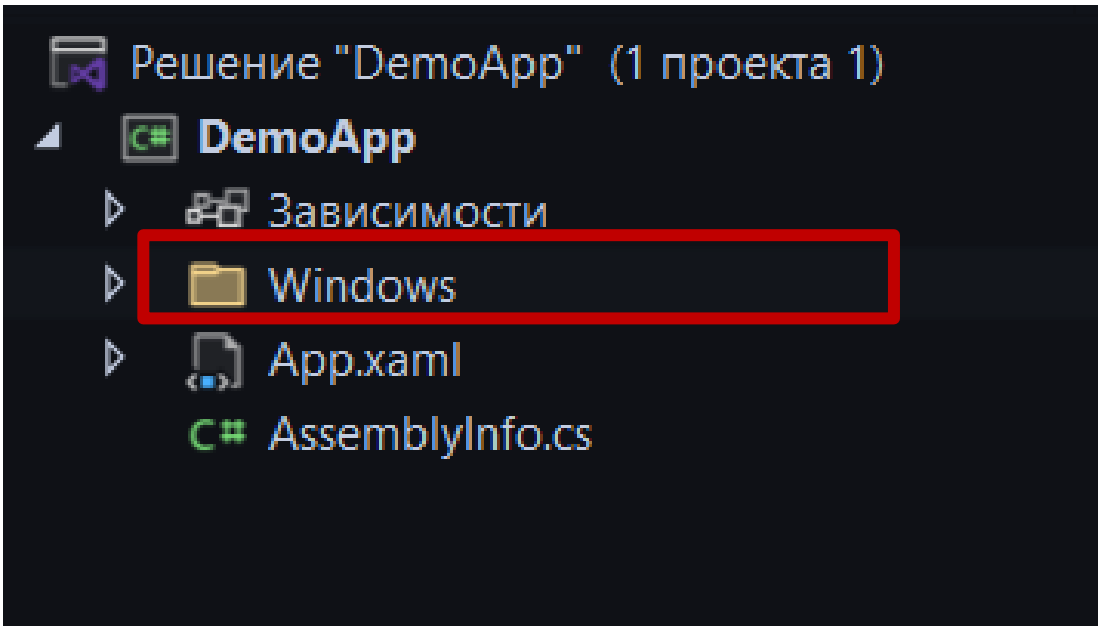
```
namespace WpfApp4
{
    Ссылок: 0
    public static class CurrentUser
    {
    }
}
```

В классе пропишите статические поля

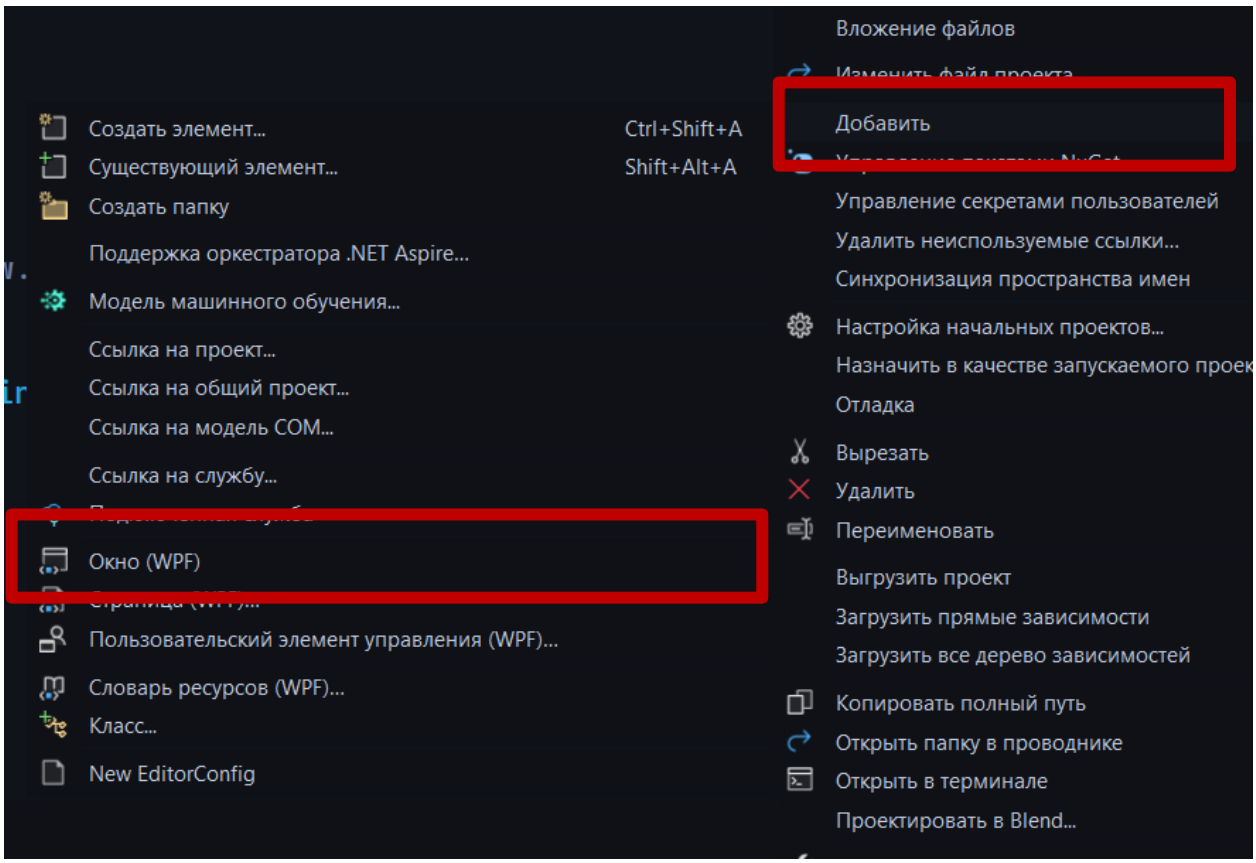
```
public static int Id { get; set; }
public static string FullName { get; set; }
public static string Role { get; set; }
```

3. Создайте новое окно

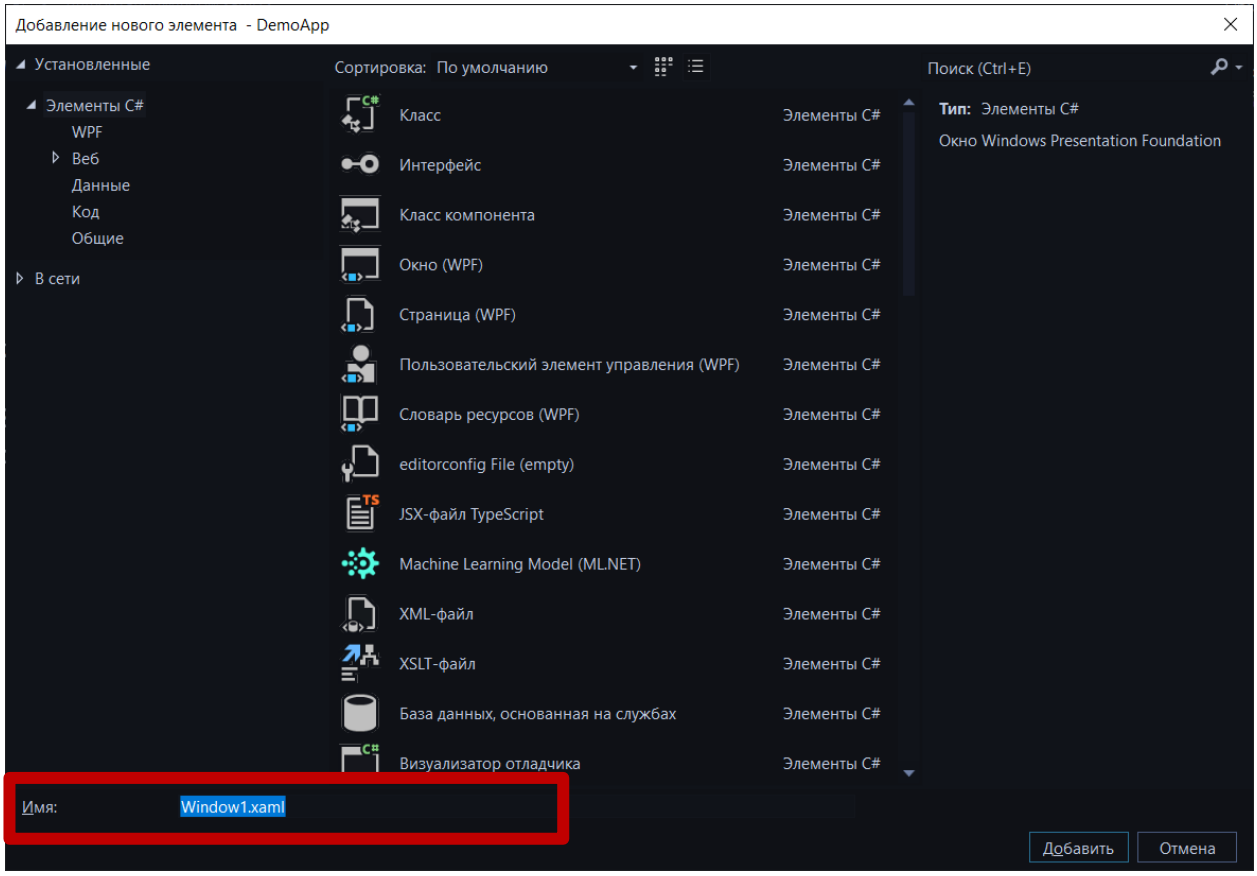
3.1 Откройте обозреватель решений, кликните ПКМ (правой кнопкой мыши по папке)



Выберите «Добавить» → «Окно»


Создать элемент... Ctrl+Shift+A
Существующий элемент... Shift+Alt+A
Создать папку
Поддержка оркестратора .NET Aspire...
Модель машинного обучения...
Ссылка на проект...
Ссылка на общий проект...
Ссылка на модель COM...
Ссылка на службу...
Окно (WPF)
Пользовательский элемент управления (WPF)...
Словарь ресурсов (WPF)...
Класс...
New EditorConfig
Вложение файлов
Добавить
Управление секретами пользователей
Удалить неиспользуемые ссылки...
Синхронизация пространства имен
Настройка начальных проектов...
Назначить в качестве запускаемого проек
Отладка
Вырезать
Удалить
Переименовать
Выгрузить проект
Загрузить прямые зависимости
Загрузить все дерево зависимостей
Копировать полный путь
Открыть папку в проводнике
Открыть в терминале
Проектировать в Blend...

Переименуйте окно, например **CatalogWindow**



(НЕ ОСТАВЛЯЙТЕ WINDOW1 И ТД, ДАВАЙТЕ ПРАВИЛЬНЫЕ НАЗВАНИЯ ДЛЯ ФАЙЛОВ И КЛАССОВ)

4. Вернемся в окно авторизации. Для удобства можно создать отдельный метод для перехода на окно с товарами

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СЕБЯ

```
private void OpenProdWin()
    {
        CatalogWindow mn = new();  // создание объекта окна
        mn.Show();          // открытие окна
        Close();            // закрытие текущего окна
    }
```

5. В методе для просмотра каталога в роли гостя (Click для кнопки и MouseLeftButtonUp для текстового блока) обратитесь к классу CurrentUser и

установите Id = 0 и все остальные поля на «Гость». После вызовите метод открытия окна с каталогом.

```
CurrentUser.Id = 0;
CurrentUser.FullName = "Гость";
CurrentUser.Role = "Гость";

OpenWindow();
```

!!!ПРИМЕР!!!

6. Теперь реализуем кнопку авторизации

В обработчике событий для кнопки входа создадим переменные (используйте неявную типизацию var) и присвоим им данные, введенные в поля для ввода, для этого необходимо использовать:

```
Текстовое_поле.Text.Trim();
```

Text — считывает текст из поля;

Trim() — является необязательным элементом, он просто убирает пробелы в конце.

Для PassworBox:

```
Текстовое_поле.Password.Trim();
```

Не забудьте обработать исключение и вывести сообщение для пользователя, если строки будут пустыми.

(string.IsNullOrEmpty(переменная)) ВСЕГДА ПРОПИСЫВАЕТСЯ С ПОМОЩЬЮ IF

```
if(string.IsNullOrEmpty(переменнаяЛогина) ||
string.IsNullOrEmpty(переменнаяПароля))
{
   MessageBox.Show("Заполните все поля");
  return;
}
```

!!!ПРИМЕР!!!

RETURN ПРОПИСЫВАТЬ ОБЯЗАТЕЛЬНО, ЧТОБЫ ПРОГРАММА ДАЛЬШЕ НЕ ВЫПОЛНЯЛА ДЕЙСТВИЯ

Далее нам нужно использовать подключение к БД:

```
using var db = new контекстБД();
```

using откроет и закроет подключение к БД.

Создаем пользователя и считываем из БД роль, почту и пароль, используйте **LINQ – запросы**:

```
var user = db.Users
.Include(u => u.Role)
.FirstOrDefault(u => u.Email == email && u.Passw == passw);
```

!!!ПРИМЕР!!! АДАПТИРУЙТЕ ПОД СВОИ ТАБЛИЦЫ

Здесь мы с помощью Include используем связанную таблицу Ролей, а с помощью FirsOrDefault выбираем первое совпадение с логином и паролем (***мы выбираем пользователя, у которого логин совпадает с введенным, а также пароль совпадает с введенным***).

Обработайте исключение, если логин и пароль не совпадет. FirsOrDefault возвращает null, если совпадения не найдены. (if (user == null)),

если пользователь все таки не найден, то выведите MessageBox, с соответствующим сообщением и пропишите return.

Далее CurrentUser присвойте айди, имя и роль.

Для имени пользователя используйте f-строки:

$«{user.ПолеФаимилия} {user.ПолеИмя} {user.ПолеОтчество}»

Для поля роли пользователя сначала обратитесь к таблице ролей, а после к полю с названием роли (user.Role.RoleName)

Вызовите метод открытия окна каталога.